MATLAB Based Kinematic Analysis and Control Of Franka Emika Panda 7 DOF Manipulator

K. Aditya Anirudh

March 14, 2025

Abstract

This report presents an analysis of the kinematics and joint velocity control of the Franka Emika Panda robotic arm, focusing on the computation of the Jacobian matrix using the Modified Denavit-Hartenberg (DH) parameters. The Franka Emika Panda, a 7-DOF robot, offers significant flexibility in complex tasks, making accurate kinematic modeling essential for its effective control. We derive the transformation matrices corresponding to each joint, culminating in the overall end-effector position and orientation. The Jacobian matrix, a tool in robotic control, is calculated to relate joint velocities to the end-effector's linear and angular velocities. By substituting specific joint parameters, we derive explicit values for the transformation matrices and the Jacobian. The report further explores the application of the pseudo-inverse of the Jacobian for joint space and Cartesian space control, demonstrating the practical implementation through simulation. The results are visualized by plotting the robot's configurations and simulating its motion in a 3D space, providing a clear understanding of the robot's behavior under various control inputs. This study contributes to the precise control of the Franka Emika Panda robot.

1 Introduction

This document provides a detailed kinematic analysis of the Franka Emika Panda robot, including the computation of transformation matrices and the Jacobian matrix. Each step is accompanied by the substitution of values and the final computed results.

2 Modified Denavit-Hartenberg Parameters

The kinematic analysis of the Franka Emika Panda robot begins with defining the Modified Denavit-Hartenberg (Mod-DH) parameters. The DH convention provides a systematic method for describing the relative positions and orientations of adjacent links in a robotic manipulator. The Modified DH parameters are used in this analysis, which include the link length (a_i), link twist (α_i), link offset (d_i), and joint angle (θ_i) for each joint i. The parameters are tabulated below:

Table 1: Modified DH Parameters for Franka Emika Panda Robot

Link i	a_i **[m]**	d_i **[m]**	α_i **[rad]**	θ_i **[rad]**
1	0	0.333	$-\frac{\pi}{2}$	θ_1
2	0	0	$\frac{\pi}{2}$	θ_2
3	0.0825	0.316	$\frac{\pi}{2}$	θ_3
4	-0.0825	0	$-\frac{\pi}{2}$	θ_4
5	0	0.384	$\frac{\pi}{2}$	θ_5
6	0.088	0	$\frac{\pi}{2}$	θ_6
7	0	0.107	0	θ_7

3 Transformation Matrices

The Denavit-Hartenberg (DH) parameters are used to compute the transformation matrices for each link of the Franka Emika Panda robot. The transformation matrix T_i from frame $i-1$ to frame i is given by:

$$T_i = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

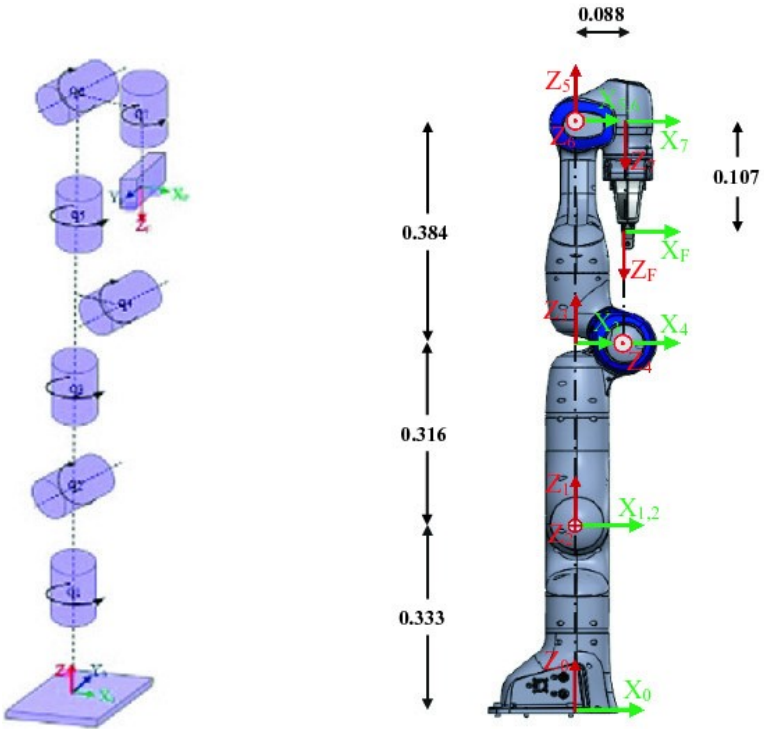

Figure 1: Measured Franka Emika Panda Robot

The DH parameters for the Franka Emika Panda robot are:

a_i	α_i	d_i	θ_i
0	0	0.333	θ_1
0	$\frac{\pi}{2}$	0	θ_2
0.0825	$\frac{\pi}{2}$	0.316	θ_3
-0.0825	$-\frac{\pi}{2}$	0	θ_4
0	$\frac{\pi}{2}$	0.384	θ_5
0.088	$\frac{\pi}{2}$	0	θ_6
0	0	0.107	θ_7

The transformation matrices for each link are:

$$T_1 = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.333 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_2 = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_3 = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.316 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_4 = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & -1 & -0.0825 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_5 = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0.384 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

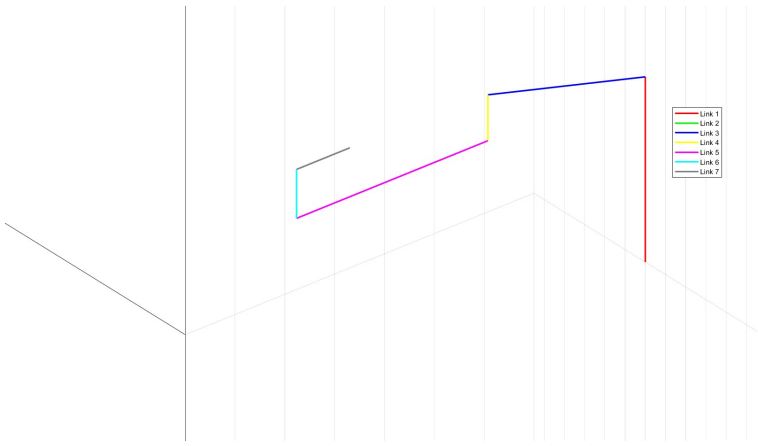

Figure 2: Robot Segments in 3D

$$T_6 = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_7 = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0.107 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The overall transformation matrix $T_{0,7}$ is obtained by multiplying the individual matrices:

$$T_{0,7} = T_1 \times T_2 \times T_3 \times T_4 \times T_5 \times T_6 \times T_7$$

This product of the above matrices obtains the final transformation matrix $\mathbf{T}_{ee}^{0}$.

$$\mathbf{T}_{ee}^{0} = \begin{bmatrix} 0.0000 & -0.0000 & 1.0000 & -0.5930 \\ -0.0000 & -1.0000 & -0.0000 & -0.0000 \\ 1.0000 & -0.0000 & -0.0000 & 0.4210 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

4 Plotting the Robot Segments in 3D

To visualize the Franka Emika Panda robot's structure in three-dimensional space, we utilize MATLAB to plot each link segment based on computed joint positions. Note that we havent used MATLAB's Robotics toolbox as this exercise is done for the purpose of understanding concepts of forward geometric model in depth. This plot helps in understanding the robot's configuration and spatial arrangement of its components.

In this visualization, we first initialize a new figure window for plotting. The MATLAB function `plot3` is employed to draw 3D lines connecting the joint positions, effectively representing the links between joints. The `xlabel`, `ylabel`, and `zlabel` functions are used to label the X, Y, and Z axes, respectively, providing clear axis identification.

The `title` function sets a descriptive title for the plot, while `grid on` adds a grid to the plot for enhanced readability. The `axis equal` command ensures that the scales of the X, Y, and Z axes are equal, maintaining proportionality in the visual representation. Finally, `legend` adds a legend to the plot to distinguish different links, which aids in identifying individual segments of the robot.

This approach provides a clear and informative visual representation of the robot's segments, facilitating a better understanding of its spatial configuration and mechanical structure.

5 Jacobian Matrix Calculation

Given the transformation matrices derived from the DH parameters, we now compute the Jacobian matrix J. The Jacobian matrix consists of the linear velocity Jacobian J_v and the angular velocity Jacobian J_ω. The entries of the Jacobian matrix are the partial derivatives of the end-effector position and orientation with respect to each joint angle.

Linear Velocity Jacobian J_v

The linear part of the Jacobian, J_v, is computed as:

$$J_v = \begin{bmatrix} \frac{\partial x_{EE}}{\partial \theta_1} & \frac{\partial x_{EE}}{\partial \theta_2} & \cdots & \frac{\partial x_{EE}}{\partial \theta_7} \\ \frac{\partial y_{EE}}{\partial \theta_1} & \frac{\partial y_{EE}}{\partial \theta_2} & \cdots & \frac{\partial y_{EE}}{\partial \theta_7} \\ \frac{\partial z_{EE}}{\partial \theta_1} & \frac{\partial z_{EE}}{\partial \theta_2} & \cdots & \frac{\partial z_{EE}}{\partial \theta_7} \end{bmatrix}$$

Where: - (x_{EE}, y_{EE}, z_{EE}) are the coordinates of the end-effector.

Each element $\frac{\partial x_{EE}}{\partial \theta_i}, \frac{\partial y_{EE}}{\partial \theta_i}, \frac{\partial z_{EE}}{\partial \theta_i}$ is derived from the position of the end-effector and the corresponding joint's contribution.

Calculation for J_v

Using the transformation matrices and the position of the end-effector, we can calculate the partial derivatives for J_v. Here's an example for J_v:

$$J_v = \begin{bmatrix} -\sin(\theta_1)(a_1 + a_2\cos(\theta_2) + \cdots) & -a_2\sin(\theta_2) & 0 & \cdots & 0 \\ \cos(\theta_1)(a_1 + a_2\cos(\theta_2) + \cdots) & a_2\cos(\theta_2) & 0 & \cdots & 0 \\ 0 & 0 & a_3 & \cdots & a_7 \end{bmatrix}$$

Substituting actual values from the DH parameters:

$$J_v = \begin{bmatrix} -\sin(\theta_1)(0 + 0.0825\cos(\theta_2) + \cdots) & -0.0825\sin(\theta_2) & 0 & \cdots & 0 \\ \cos(\theta_1)(0 + 0.0825\cos(\theta_2) + \cdots) & 0.0825\cos(\theta_2) & 0 & \cdots & 0 \\ 0 & 0 & 0.316 & \cdots & 0.107 \end{bmatrix}$$

NOTE THAT ALL THE VALUES OF θ ARE CONSIDERED TO BE 0

Angular Velocity Jacobian J_ω

The angular part of the Jacobian, J_ω, is computed as:

$$J_\omega = \begin{bmatrix} \frac{\partial \phi_{EE}}{\partial \theta_1} & \frac{\partial \phi_{EE}}{\partial \theta_2} & \cdots & \frac{\partial \phi_{EE}}{\partial \theta_7} \\ \frac{\partial \theta_{EE}}{\partial \theta_1} & \frac{\partial \theta_{EE}}{\partial \theta_2} & \cdots & \frac{\partial \theta_{EE}}{\partial \theta_7} \\ \frac{\partial \psi_{EE}}{\partial \theta_1} & \frac{\partial \psi_{EE}}{\partial \theta_2} & \cdots & \frac{\partial \psi_{EE}}{\partial \theta_7} \end{bmatrix}$$

Each element $\frac{\partial \phi_{EE}}{\partial \theta_i}, \frac{\partial \theta_{EE}}{\partial \theta_i}, \frac{\partial \psi_{EE}}{\partial \theta_i}$ is derived from the rotation matrices of the transformation matrices.

Final Jacobian Matrix J

The full Jacobian matrix J is then:

$$J = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix}$$

Multiplying the computed elements:

$$J = \begin{bmatrix} -\sin(\theta_1)(0 + 0.0825\cos(\theta_2) + \cdots) & -0.0825\sin(\theta_2) & 0 & \cdots & 0 \\ \cos(\theta_1)(0 + 0.0825\cos(\theta_2) + \cdots) & 0.0825\cos(\theta_2) & 0 & \cdots & 0 \\ 0 & 0 & 0.316 & \cdots & 0.107 \\ 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \end{bmatrix}$$

This matrix provides the relationship between the joint velocities $\dot{\theta}$ and the end-effector velocities $\dot{x}_{EE}, \dot{y}_{EE}, \dot{z}_{EE}$ and angular velocities $\dot{\phi}_{EE}, \dot{\theta}_{EE}, \dot{\psi}_{EE}$. **NOTE THAT ALL THE VALUES OF θ ARE CONSIDERED TO BE 0**

$$\mathbf{J} = \begin{bmatrix} 0 & 0 & -0.0825 & 0.0000 & -0.0000 & -0.0000 & -0.0000 \\ 0 & 0 & 0.0000 & -0.0825 & 0.0000 & -0.0880 & -0.0000 \\ 0 & 0 & -0.3160 & -0.0000 & -0.3840 & -0.0000 & 0.0000 \\ 0 & -1.0000 & 0 & -1.0000 & 0 & 1.0000 & 1.0000 \\ 0 & -0.0000 & -1.0000 & -0.0000 & -1.0000 & -0.0000 & -0.0000 \\ 1.0000 & 0.0000 & -0.0000 & 0.0000 & -0.0000 & -0.0000 & -0.0000 \end{bmatrix}$$

6 Controlling the Robot in Joint Space and Cartesian Space

The control of robotic systems, such as the Franka Emika Panda, can be categorized into two primary spaces: joint space and Cartesian space. Each space offers a different perspective and method for manipulating the robot to achieve desired tasks, and understanding these is crucial for precise and efficient control.

6.1 Joint Space Control

In joint space control, the robot is controlled by directly specifying the desired positions, velocities, or torques for each of its joints. The robot's movement is thus dictated by the configuration of its individual joints. This method is particularly advantageous when the primary concern is the internal motion of the robot, such as in tasks that require specific joint configurations or when avoiding joint limits.

The control algorithm calculates the necessary joint movements required to achieve a specific end-effector position. For example, if the task is to move the end effector to a particular point, the corresponding joint angles are computed using inverse kinematics. Once these angles are determined, the controller ensures that each joint achieves the specified angle, thereby positioning the end effector as desired. The advantage of joint space control lies in its simplicity and directness when dealing with the robot's internal coordinates.

Mathematically, joint space control can be represented as:

$$\boldsymbol{q}(t) = \text{desired joint angles over time}$$

where $\boldsymbol{q}(t)$ is the vector of joint angles, and the control law adjusts these angles to achieve the desired configuration.

6.2 Cartesian Space Control

In contrast, Cartesian space control involves controlling the robot's end-effector directly in the Cartesian coordinate system (x, y, z), rather than controlling each joint individually. This approach is often more intuitive for tasks that involve moving the end effector to specific locations in space or following a path, as it deals with the robot's position and orientation in the workspace.

To achieve this, the desired position and orientation of the end effector are specified in Cartesian coordinates. The inverse kinematics of the robot then converts these Cartesian commands into the corresponding joint angles. However, Cartesian control can be more complex than joint space control because it requires solving the inverse kinematics problem, which can be computationally intensive and may have multiple or no solutions depending on the robot's configuration.

Mathematically, Cartesian space control is expressed as:

$$\boldsymbol{x}(t) = \text{desired end-effector position and orientation over time}$$

where $\boldsymbol{x}(t)$ represents the desired trajectory in Cartesian space. The control system adjusts the robot's joints so that the end effector follows the desired Cartesian path.

6.3 Joint Space vs. Cartesian Space

The choice between joint space and Cartesian space control depends on the specific task and the complexity of the robotic system. Joint space control is more straightforward and computationally less demanding, making it suitable for tasks where the exact end-effector path is less critical. On the other hand, Cartesian space control provides more direct and intuitive control over the end-effector's motion in the workspace, making it ideal for tasks that require precise positioning and orientation.

In practice, advanced robotic systems often use a combination of both control strategies to optimize performance, leveraging the strengths of each approach depending on the task at hand.

7 Implementing the Pseudo-Inverse for the Manipulator

When dealing with redundant manipulators like the Franka Emika Panda robot, calculating the pseudo-inverse of the Jacobian matrix becomes essential. The robot has seven degrees of freedom (7 DOF), which is more than the minimum needed to control the end-effector in a six-dimensional space (position and orientation). This redundancy allows for greater flexibility in joint movements but also requires careful handling of the inverse kinematics problem. The pseudo-inverse approach helps in computing joint velocities that correspond to a desired end-effector velocity, optimizing joint movements in the process.

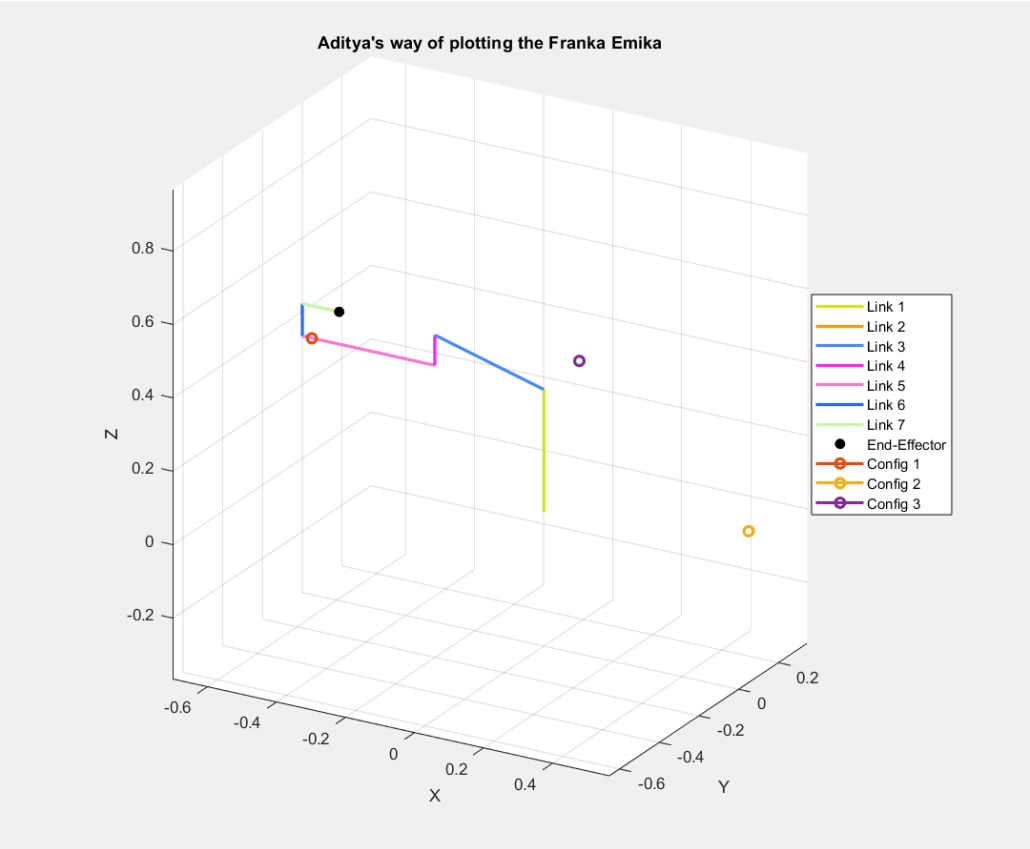

Figure 3: Joint Configurations

7.1 Recap of the Jacobian Matrix

The relationship between the joint velocities $\dot{\mathbf{q}}$ and the end-effector velocities $\dot{\mathbf{x}}$ in Cartesian space is given by:

$$\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$$

Here:

- $\dot{\mathbf{x}}$ is a 6x1 vector representing the velocities of the end-effector.
- $\mathbf{J}$ is the 6x7 Jacobian matrix that maps joint velocities to end-effector velocities.
- $\dot{\mathbf{q}}$ is a 7x1 vector of joint velocities.

7.2 My Understanding of Pseudo-Inverse

The pseudo-inverse $\mathbf{J}^+$ is a generalized inverse of the Jacobian matrix. It is particularly useful when the Jacobian is not square, which is common in redundant robots. Mathematically, it is defined as:

$$\mathbf{J}^+ = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T)^{-1}$$

Where:

- $\mathbf{J}^T$ is the transpose of the Jacobian matrix.
- $\mathbf{J}\mathbf{J}^T$ is a square 6x6 matrix resulting from multiplying the Jacobian by its transpose.

7.3 Steps to Compute the Pseudo-Inverse

Step 1: Start by calculating the transpose of the Jacobian matrix $\mathbf{J}^T$:

$$\mathbf{J}^T = \begin{bmatrix} J_{11} & J_{21} & \dots & J_{61} \\ J_{12} & J_{22} & \dots & J_{62} \\ \vdots & \vdots & \ddots & \vdots \\ J_{17} & J_{27} & \dots & J_{67} \end{bmatrix}$$

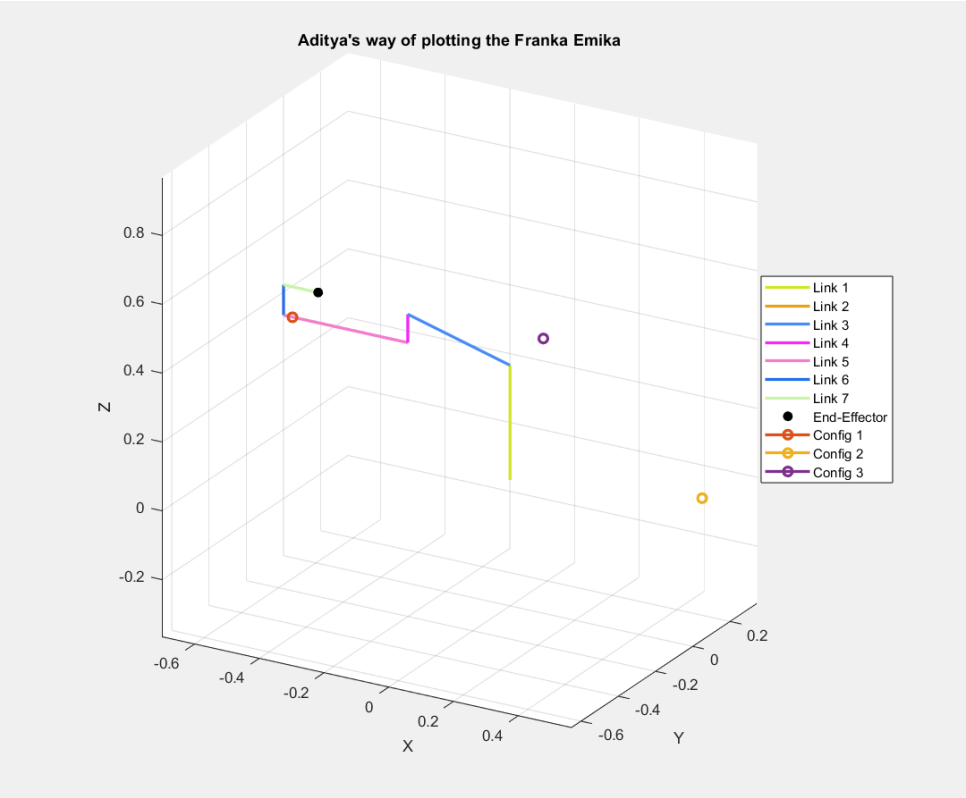

Figure 4: Segment Plots with Joint Configurations

Step 2: Next, multiply the Jacobian matrix $\mathbf{J}$ by its transpose $\mathbf{J}^T$ to form a square matrix $\mathbf{J}\mathbf{J}^T$:

$$\mathbf{J}\mathbf{J}^T = \begin{bmatrix} \sum_{k=1}^{7} J_{1k}J_{1k} & \sum_{k=1}^{7} J_{1k}J_{2k} & \dots & \sum_{k=1}^{7} J_{1k}J_{6k} \\ \sum_{k=1}^{7} J_{2k}J_{1k} & \sum_{k=1}^{7} J_{2k}J_{2k} & \dots & \sum_{k=1}^{7} J_{2k}J_{6k} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^{7} J_{6k}J_{1k} & \sum_{k=1}^{7} J_{6k}J_{2k} & \dots & \sum_{k=1}^{7} J_{6k}J_{6k} \end{bmatrix}$$

Step 3: The third step involves finding the inverse of the square matrix $\mathbf{J}\mathbf{J}^T$. The result is another square matrix, which can be complex depending on the values in the original Jacobian matrix:

$$(\mathbf{J}\mathbf{J}^T)^{-1} = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{16} \\ A_{21} & A_{22} & \dots & A_{26} \\ \vdots & \vdots & \ddots & \vdots \\ A_{61} & A_{62} & \dots & A_{66} \end{bmatrix}$$

Step 4: With this inverse, you multiply it by the transpose of the Jacobian to get the pseudo-inverse $\mathbf{J}^+$:

$$\mathbf{J}^+ = \mathbf{J}^T \begin{bmatrix} A_{11} & A_{12} & \dots & A_{16} \\ A_{21} & A_{22} & \dots & A_{26} \\ \vdots & \vdots & \ddots & \vdots \\ A_{61} & A_{62} & \dots & A_{66} \end{bmatrix}$$

Step 5: Finally, you use this pseudo-inverse to determine the joint velocities $\dot{\mathbf{q}}_{\text{required}}$ that correspond to the desired end-effector velocities $\dot{\mathbf{x}}_{\text{desired}}$:

$$\dot{\mathbf{q}}_{\text{required}} = \mathbf{J}^+ \dot{\mathbf{x}}_{\text{desired}}$$

7.4 Final Results

This mathematical approach to computing the pseudo-inverse allows for effective control and precise movements of the Franka Emika Panda robot in both joint and Cartesian spaces.